

VIRTUALAB DIGITAL · HANDS-ON PRACTICE

Safety Parameter Learning Guide

Step-by-step implementation guide for seven core API safety patterns. Copy the code, follow the exercises, and build production-ready security into your own projects. Framework-agnostic examples in Node.js and Python.

Author: Z.ai · **Date:** August 2026

Patterns: 7 · **Exercises:** 6 · **Level:** Beginner to Intermediate

PRACTICE · BUILD · DEPLOY

Table of Contents

h_a42b9a43	3
h_f2672e8b	3
h_e845e46a	3
h_b6487b6f	3
h_ccff6a9d	3
h_902e38ad	4
h_51ec099e	4
h_57cbf86e	4
h_a6f1efcc	4
h_d2a77780	4
h_6fea9130	5
h_d3120b40	5
h_beb335a8	5
h_219a0feb	5
h_f51bc04e	6
h_b9de0c3c	6
h_61be0afd	6
h_090d29ba	6
h_bc0d5b4c	6

h_7567d513 **7**

h_12cd92287

h_92404dc27

h_dad975dc **7**

h_89c3b21f7

h_11612cff8

h_a65ab2dc **8**

h_fd1c7ebd8

h_21fb76fd8

h_1a53a6b19

Chapter 1: Getting Started

1.1 What You Will Learn

This guide teaches you how to implement the seven core API safety patterns used in Eli, but adapted for any personal project. You do not need Eli or a Next.js application to follow along. The patterns are framework-agnostic and can be implemented in Express, FastAPI, Flask, or plain Node.js. Each chapter explains the concept, shows a minimal implementation you can copy into your own project, and ends with an exercise to deepen your understanding. The patterns build on each other, so it is recommended to work through them in order.

1.2 Prerequisites

You need basic familiarity with any server-side language (JavaScript/TypeScript, Python, or Go) and HTTP concepts (status codes, headers, request/response lifecycle). No security expertise is assumed. Each pattern starts with the problem it solves, so even if you have never implemented an auth check before, you will understand why it exists before you write the code.

Pattern	Difficulty	Time	What You Build
1. Auth Gate	Beginner	10 min	Bearer token check that protects any endpoint
2. Rate Limiter	Beginner	20 min	Sliding-window rate limiter per IP address
3. Payload Limits	Beginner	5 min	Content-length check that rejects oversized requests
4. Input Sanitization	Intermediate	15 min	Text normalizer that strips dangerous characters
5. Prompt Injection Blocker	Intermediate	20 min	Pattern matcher that detects LLM manipulation attempts
6. Key Validation	Intermediate	15 min	Format checker for API keys before they are used
7. Audit Logger	Beginner	15 min	Structured event logger with file persistence

Table 1: The seven patterns, difficulty, and what you will build in each exercise.

Chapter 2: Pattern 1 - Auth Gate

2.1 The Problem

Any API endpoint accessible on the internet can be discovered by scanners, bots, or curious users. Without authentication, every endpoint is open to anyone who guesses or discovers the URL. For a project that consumes paid or rate-limited APIs (like LLM services), an open endpoint means anyone can drain your API budget. The auth gate is the simplest and most effective protection against this class of threat. It does not need

to be complex: a single shared secret checked on every request is sufficient for most personal projects and small teams.

2.2 Implementation (Node.js / Express)

The implementation checks two sources: the `Authorization` header (standard for all HTTP clients) and a `?key=` query parameter (convenient for quick curl tests). If no master key is configured, the gate is disabled, which is the right default for local development. Copy this into a file called `auth.js`:

```
function checkAuth(req) { const masterKey = process.env.API_KEY; if (!masterKey) return true; // Auth disabled const authHeader = req.headers['authorization']; if (authHeader === `Bearer ${masterKey}`) return true; const queryKey = req.query?.key; if (queryKey === masterKey) return true; return false; } // Usage in Express route: app.post('/api/chat', (req, res) => { if (!checkAuth(req)) { return res.status(401).json({ error: 'Unauthorized' }); } // ... your handler });
```

2.3 Implementation (Python / FastAPI)

```
from fastapi import Request, HTTPException import os def check_auth(request: Request) -> bool: master_key = os.getenv('API_KEY') if not master_key: return True # Auth disabled auth_header = request.headers.get('authorization', '') if auth_header == f'Bearer {master_key}': return True query_key = request.query_params.get('key') if query_key == master_key: return True return False # Usage: @app.post('/api/chat') async def chat(request: Request): if not check_auth(request): raise HTTPException(401, 'Unauthorized')
```

Exercise: Exercise: Wire the auth gate into two endpoints. One should require auth, the other should be public (like a health check). Test with curl using both the header method and the query parameter method. Verify that a request with the wrong key gets a 401 response.

Tip: Production tip: Never commit your API_KEY to source control. Use a .env file (add it to .gitignore) or set it via your hosting platform environment variables panel.

Chapter 3: Pattern 2 - Rate Limiter

3.1 The Problem

Even authenticated users can make too many requests. A misconfigured client, a buggy retry loop, or a deliberate attack can send hundreds of requests per second. Rate limiting caps the number of requests a single client can make within a time window. The sliding window algorithm is preferred over fixed windows because it prevents burst-at-boundary abuse, where a client sends all their allowed requests in the last millisecond of one window and the first millisecond of the next, effectively getting double the intended rate.

3.2 Implementation

This is a framework-agnostic implementation using a plain Map. It stores an array of timestamps per IP, prunes old entries on each check, and compares the count against the configured maximum. The cleanup runs every 5

minutes to prevent memory leaks from stale entries. Copy this into `rateLimiter.js`:

```
const windows = new Map(); // ip -> [timestamps] let lastCleanup = Date.now(); function
checkRateLimit(ip, maxRequests, windowMs) { const now = Date.now(); // Cleanup stale entries
every 5 minutes if (now - lastCleanup > 300_000) { for (const [key, ts] of windows) { const
cutoff = now - windowMs; ts.array = ts.array.filter(t => t > cutoff); if (ts.array.length
=== 0) windows.delete(key); } lastCleanup = now; } let entry = windows.get(ip); if (!entry)
{ entry = { array: [] }; windows.set(ip, entry); } // Prune timestamps outside the window
const cutoff = now - windowMs; entry.array = entry.array.filter(t => t > cutoff); if
(entry.array.length >= maxRequests) return false; entry.array.push(now); return true; } //
Usage in Express: const ip = req.ip || req.headers['x-forwarded-for']?.split(',')[0]; if
(!checkRateLimit(ip, 15, 60_000)) { return res.status(429).json({ error: 'Too many requests'
}); }
```

The key design decisions in this implementation: rejected requests do NOT consume a slot (the timestamp is only pushed when the request is allowed), the cleanup is $O(n)$ over all tracked IPs but runs infrequently, and the window slides smoothly rather than resetting at fixed intervals. These decisions prevent both the burst-at-boundary problem and the memory leak problem.

Exercise: Implement the rate limiter and test it with a loop that sends 20 rapid requests. Verify that exactly 15 succeed and the remaining 5 get 429 responses. Then wait 60 seconds and verify that requests succeed again.

Chapter 4: Pattern 3 - Payload Limits

4.1 The Problem

An attacker can send a request with a Content-Length of 2GB. If your server tries to parse that into memory, it will crash with an out-of-memory error, taking down your entire application. The fix is trivially simple: check the Content-Length header before parsing the body. This check costs almost nothing (one integer comparison) but prevents one of the most effective denial-of-service attacks against JSON APIs.

4.2 Implementation

```
// Middleware for Express: function payloadLimit(maxBytes) { return (req, res, next) => {
const len = parseInt(req.headers['content-length'] || '0', 10); if (len > maxBytes) { return
res.status(413).json({ error: 'Payload too large' }); } next(); }; } // Apply to routes:
app.post('/api/chat', payloadLimit(10_240), (req, res) => { // Body is guaranteed to be
under 10KB }); // Python equivalent: @app.post('/api/chat') async def chat(request:
Request): cl = int(request.headers.get('content-length', 0)) if cl > 10240: raise
HTTPException(413, 'Payload too large')
```

Tip: Choose payload limits based on your expected input size. Chat messages: 10KB. File uploads: check your framework built-in limits. Configuration updates: 4KB. The key insight is that every route should have a limit tuned to its expected use case, plus an absolute maximum (like 100KB) that no route can exceed.

Chapter 5: Pattern 4 - Input Sanitization

5.1 The Problem

User input cannot be trusted. Even well-intentioned users may paste content that contains invisible control characters, null bytes, or Unicode normalization attacks. A null byte ( ) in a string can cause truncation in downstream C libraries. Different Unicode representations of the same character can bypass regex filters (homoglyph attacks). Excessively long inputs can bypass length checks that only look at character count without considering encoding expansion. The sanitizer handles all of these in a single pass.

5.2 Implementation

```
function sanitizeInput(text, maxLength = 4000) { if (!text || typeof text !== 'string')
return ''; // 1. Remove null bytes (prevents truncation attacks) let clean =
text.replace(/\x00/g, ''); // 2. Normalize Unicode (prevents homoglyph bypass) clean =
clean.normalize('NFC'); // 3. Collapse excessive whitespace clean = clean.replace(/\n{4,}/g,
'\n\n\n'); clean = clean.replace(/ {4,}/g, ' '); // 4. Remove control characters (keep , , )
clean = clean.replace(/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]/g, ''); // 5. Trim and enforce max
length clean = clean.trim(); if (clean.length > maxLength) clean = clean.slice(0,
maxLength); return clean; }
```

Exercise: Create a test suite with the following inputs and verify the sanitizer handles each correctly: (1) a string with embedded null bytes, (2) a string with (5 newlines), (3) a string with a control character like  , (4) a string that is 10,000 characters long (should be truncated to your maxLength), (5) a string with Unicode homoglyphs (different representations of the same character).

Chapter 6: Pattern 5 - Prompt Injection Blocker

6.1 The Problem

If your application sends user input to an LLM, you are vulnerable to prompt injection. An attacker can craft input that overrides your system prompt, instructs the model to ignore previous instructions, or manipulates the output format. This is not a theoretical risk: prompt injection is the most common attack vector against LLM-powered applications. The blocker uses regex pattern matching to detect the most common injection attempts before they reach the model. It is not a replacement for good system prompt engineering, but it provides defense-in-depth.

6.2 Implementation

```
const INJECTION_PATTERNS = [
/ignore\s+(all\s+)?(previous|above|prior)\s+(instructions?|prompts?|system)/i,
/you\s+are\s+now\s+(a|an|the)\s+/i, /system\s*:\s*$/im, /---\s*(END|STOP|FINISH)\s*---/i,
/\[INST\]\[\[/INST\]/i, /<\|im_start\|>|<\|im_end\|>/i,
/respond\s+(only\s+)?(with|in)\s+(json|yaml|xml|code|markdown)/i,
/forget\s+(everything|all|your)\s+(instructions?|training|rules)/i,
/pretend\s+(you\s+are|to\s+be)/i, /act\s+as\s+(if\s+)?(you\s+)?(a|an|the)\s+/i,
/jailbreak/i, /DAN\s+mode/i, ]; function checkPromptInjection(text) { for (const pattern of
INJECTION_PATTERNS) { if (pattern.test(text)) { return { blocked: true, pattern:
pattern.source }; } } return { blocked: false }; } // Usage: const result =
```

```
checkPromptInjection(userMessage); if (result.blocked) { return res.status(400).json({
error: 'Message contains patterns that look like prompt injection.', detected: true, }); }
```

Tip: Design choice: flag-only vs. block. In Eli, we chose to block detected injections (return 400). An alternative is to flag but allow (log the detection and proceed). Flag-only is appropriate when you trust your system prompt to resist injection and do not want false positives to annoy users. Block is appropriate when the cost of a successful injection is high (e.g., the LLM has access to sensitive data or can execute actions). Choose based on your threat model.

Chapter 7: Pattern 6 - Key Validation

7.1 The Problem

If your application accepts API keys from users (for example, a multi-model LLM proxy where users provide their own keys), you must validate the key format before using it. A malformed key will cause an API error that wastes a network round-trip and may expose error details. A key that is too long could be an injection attempt against your system. The validator checks format, length, and pattern before the key is stored or used.

7.2 Implementation

```
const KEY_PATTERNS = { gemini: { pattern:
/^(AIza[A-Za-z0-9_-]{33,}|AQ\.[A-Za-z0-9_-]{30,})$/, minLength: 20, }, openai: { pattern:
/^sk-[A-Za-z0-9]{20,}$/, minLength: 25, }, }; function validateKeyFormat(service, key) {
const svc = KEY_PATTERNS[service]; if (!svc) return { valid: false, reason: `Unknown
service: ${service}` }; if (!key || typeof key !== 'string') return { valid: false, reason:
'Key is empty or not a string' }; if (key.length < svc.minLength) return { valid: false,
reason: `Key too short (min ${svc.minLength})` }; if (key.length > 500) return { valid:
false, reason: 'Key too long (max 500)' }; if (!svc.pattern.test(key)) return { valid:
false, reason: `Key does not match ${service} format` }; return { valid: true, reason:
'Format valid' }; }
```

Exercise: Add a third service (e.g., Anthropic with pattern `sk-ant-...`) to the validator. Test with: (1) a valid Gemini key, (2) an invalid key that is too short, (3) a valid-looking key for an unknown service, (4) a 600-character key (should fail max length), (5) a key with SQL injection characters (should fail pattern match).

Chapter 8: Pattern 7 - Audit Logger

8.1 The Problem

When something goes wrong, you need to know what happened. `Console.log` is not enough: it disappears on restart, has no structure, and cannot be easily queried. An audit log provides a persistent, structured record of security-relevant events. It should capture who did what, when, and with what context. The implementation should be async (never block the request handler), bounded in memory (prevent memory leaks), and persisted to disk (survive restarts).

8.2 Implementation

```
const fs = require('fs'); const path = require('path'); const MAX_MEMORY = 500; const buffer = []; const LOG_FILE = path.join(process.cwd(), 'data', 'audit.jsonl'); function ensureDir() { fs.mkdirSync(path.dirname(LOG_FILE), { recursive: true }); } function audit(event, detail, meta = {}, ip) { const entry = { ts: new Date().toISOString(), event, detail, meta, ip, }; buffer.push(entry); if (buffer.length > MAX_MEMORY) buffer.shift(); // Async write - never block the caller try { ensureDir(); fs.appendFileSync(LOG_FILE, JSON.stringify(entry) + '\n'); } catch (err) { console.error('Audit write failed:', err.message); } } // Usage: audit('auth.blocked', 'Failed login from 1.2.3.4', { ip: '1.2.3.4' }, '1.2.3.4'); audit('key.rotation', 'Gemini key rotated', { service: 'gemini', inboxCount: 3 });
```

Tip: JSONL (JSON Lines) format is ideal for audit logs: each line is a valid JSON object, making it both human-readable and machine-parseable. You can grep it, pipe it to jq for filtering, or load it into any log analysis tool. Avoid binary formats or databases for audit logs: they add complexity and create dependencies that make the log harder to access during an incident.

Chapter 9: Putting It All Together

9.1 The Defense-in-Depth Stack

When you wire all seven patterns together in the correct order, you get a defense-in-depth stack where each layer filters cheaply before the next, more expensive layer runs. The ordering matters: authentication is checked first (one string comparison), then rate limiting (one map lookup), then payload size (one integer comparison), then input validation (regex operations), and finally business logic (network I/O, database queries, LLM calls). This ensures that blocked requests consume the minimum possible server resources.

Order	Pattern	Cost	Blocks
1	Auth Gate	O(1) string compare	Unauthorized access
2	Rate Limiter	O(k) array filter	Abuse, brute force, DoS
3	Payload Limit	O(1) integer compare	Memory exhaustion
4	Input Sanitization	O(n) string ops	Injection, control chars, homoglyphs
5	Prompt Injection Block	O(n) regex scan	LLM manipulation
6	Key Validation	O(1) regex test	Malformed key errors
7	Audit Logger	O(1) push + async write	(Passive: records everything)

Table 2: Complete defense-in-depth stack with per-layer cost analysis.

9.2 Wiring Example (Express)

Here is how all seven patterns combine into a single Express route handler. Notice the ordering: each check is a short-circuit return that prevents the next (more expensive) check from running if the request is already rejected.

```

app.post('/api/chat', async (req, res) => { const ip = req.ip ||
req.headers['x-forwarded-for']?.split(',')[0]; // 1. Auth if (!checkAuth(req)) {
audit('auth.blocked', `Auth failed from ${ip}`, {}, ip); return res.status(401).json({
error: 'Unauthorized' }); } // 2. Rate limit if (!checkRateLimit(ip, 15, 60_000)) {
audit('chat.ratelimited', `Rate limited from ${ip}`, {}, ip); return res.status(429).json({
error: 'Too many requests' }); } // 3. Payload const len =
parseInt(req.headers['content-length'] || '0', 10); if (len > 10_240) {
audit('chat.blocked', `Payload too large from ${ip}`, { size: len }, ip); return
res.status(413).json({ error: 'Payload too large' }); } const { message } = req.body; // 4.
Sanitize const clean = sanitizeInput(message); if (!clean) return res.status(400).json({
error: 'Empty message' }); // 5. Prompt injection if (checkPromptInjection(clean).blocked) {
audit('prompt.injection.blocked', `Blocked from ${ip}`, {}, ip); return
res.status(400).json({ error: 'Prompt injection detected' }); } // 6. Business logic (LLM
call) audit('llm.call', `Processing chat from ${ip}`, {}, ip); const response = await
callLLM(clean); return res.json({ response }); });

```

9.3 Next Steps

Once you have all seven patterns working, you have a solid Tier 1 safety foundation. The next improvements to consider are: (1) per-user rate limits instead of per-IP, which handles multiple users behind a NAT; (2) structured logging with a proper library like Pino or Winston instead of raw `appendFileSync`; (3) automated tests for each safety layer; (4) a middleware wrapper that applies all checks in one line instead of repeating them in every handler; and (5) monitoring and alerting that triggers when the rate limiter or auth gate is hit frequently, which could indicate an attack.

Exercise: Final project: Build a complete API with all seven patterns wired up. Add at least two routes with different rate limits and payload limits. Add a public health endpoint that skips auth. Test everything with curl and verify the correct HTTP status codes. Then deploy it and verify the audit log writes to disk.